

Ch. 16

現代程式開發與生成式

AI 輔助

Horazon

應用程式設計

本章目標

1. **跨越鴻溝**：從積木 (Blocks) 到代碼 (Code)。
2. **AI 輔助**：如何用 ChatGPT/Claude 寫程式。
3. **未來路徑**：App Inventor 之後可以學什麼？
4. **課程總結**：我們學會了什麼。

為什麼要學代碼 (Code) ?

App Inventor (積木) 是一個很棒的**入門工具**，它讓我們專注在「邏輯」而非「語法」。
但如果你想成為專業工程師，**文字型語言** 是必經之路。

文字代碼的優勢

1. **表達力強**：有些複雜邏輯，積木要拉很長，代碼只要兩行。
2. **工具生態**：GitHub 版控、VS Code 編輯器、各種強大的 Framework。
3. **職業需求**：業界大多使用 Java, Kotlin, Swift, JavaScript, Python。

對照表：積木 vs Python

讓我們看幾個例子，你會發現它們其實一模一樣：

變數 (Variable)

- AI2: `initialize global name to "Tom"`
- Python: `name = "Tom"`

條件 (If)

- AI2: `If score >= 60 then ...`
- Python:

```
if score >= 60:  
    print("Pass")
```

對照表：積木 vs Python

迴圈 (Loop)

- AI2: `for each number from 1 to 10`
- Python:

```
for i in range(1, 11):  
    print(i)
```

清單 (List)

- AI2: `make a list (1, 2, 3)`
- Python: `my_list = [1, 2, 3]`

結論：你在 AI2 學到的邏輯，可以直接套用到 Python ！

你的超級助手：生成式 AI

以前學程式要背語法，現在不用了。

你有 ChatGPT, Claude, Gemini !

AI 能幫你做什麼？

1. **解釋代碼**: 看不懂的程式貼給它，「請解釋這段在做什麼」。
2. **翻譯語言**: 「請把這段 App Inventor 的邏輯改寫成 Python」。
3. **除錯 (Debug)**: 「我的程式出現 Error 404，怎麼辦？」
4. **生成範例**: 「請給我一個 Python 猜數字遊戲的範例」。

實戰：如何問 AI (Prompt Engineering)

寫出好的提示詞 (Prompt)，AI 才會給你好的答案。

✗ 壞的問法

"幫我寫一個遊戲。"

(太籠統，它不知道你要什麼)

✓ 好的問法

"我是一個初學者，我想用 Python 寫一個 **剪刀石頭布** 的遊戲。

請給我完整的程式碼，並在每一行加上**中文註解**，

解釋它跟 **變數** 和 **If 判斷式** 的關係。"

實戰：用 AI 解決 AI2 的問題

如果你在 AI2 卡關了，也可以問 AI！

"我在 App Inventor 做一個藍牙連線的 App，
但是連線成功後無法傳送文字。

我的積木是 `BluetoothClient1.SendText`，
請問可能有哪些原因導致失敗？"

AI 會告訴你：

1. UUID 設定錯了？
2. 藍牙模組沒通電？
3. 權限沒開？

未來的學習地圖

如果你喜歡...

1. 繼續做 App

- Android: 學習 Kotlin (Android Studio)。
- iOS: 學習 Swift (Xcode)。
- 跨平台: 學習 Flutter (Dart) 或 React Native (JS)。

2. 喜歡邏輯與資料

- Python: 資料分析、AI 應用、網頁後端。

3. 喜歡做網頁

- HTML / CSS / JavaScript: 網頁三劍客。

課程總結

這學期我們學到了：

1. **運算思維 (Computational Thinking)**：拆解問題、模式識別、抽象化、演算法。
2. **UI/UX 設計**：Layout 排版、使用者互動。
3. **軟硬整合**：Sensor、網路、藍牙。
4. **專案實作**：從構想到實作出一個可用的 App。

工具會變 (積木 -> 代碼)，但邏輯思維是不變的。

希望大家能保持這份解決問題的能力！